

# BRAINSTORMING AND IDEATION PHASE

## Brainstorming

### Step 1: Team Gathering, Collaboration and Select the Problem Statement

#### Problem Statement

A system may contain student records with different branch values, such as ECE, EEE, and CSE. If every user can access or modify all records, sensitive information may be exposed or changed without permission. Managing access manually can also lead to inconsistent security.

#### How Might We?

How might we restrict access to student records based on the Branch field and user roles, while allowing authorized users to perform only the actions they are permitted to do?

#### Initial Discussion

The team discussed ways to protect records in ServiceNow. The main requirements identified were to restrict reading of records based on the Branch field, use roles to control access, and separately manage the ability to create, edit, and delete records. Administrators should retain the intended administrative access.

The team identified the following access-control needs:

| Access need      | Example                                                               |
|------------------|-----------------------------------------------------------------------|
| Read restriction | Users with the required role should read records where Branch is EEE. |
| Create control   | Only users with the required create permission should create records. |
| Write control    | Only authorized users should edit existing records.                   |
| Delete control   | Only authorized users should delete records.                          |

Figure 1: Main access-control requirements for the Institution Details table.

## Step 2: Brainstorm, Idea Listing, Grouping

The team generated different ideas to secure student records in ServiceNow. The ideas were grouped according to user identity, record-level restrictions, permitted actions, and testing. The team focused on a solution that could be configured and maintained using ACLs.

### Detailed Ideas

1. Create a custom table named **Institution Details** (u\_institution\_details) to store student records.
2. Add fields such as Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, and Description.
3. Create a choice field for Branch with ECE, EEE, and CSE values.
4. Create a test user and custom roles named **bb1, bb2, bb3, and bb4**.
5. Configure a Read ACL with the Branch condition set to EEE and a role requirement.
6. Use a script in the Read ACL to check for the admin and bb1 roles.
7. Create separate Create, Write, and Delete ACLs using bb2, bb3, and bb4 respectively.
8. Test the access rules by impersonating users with different role combinations.

Figure 2: Brainstormed ideas grouped around table setup, role management, ACL configuration, and validation.

### Idea Groups

| Group              | Ideas included                                              |
|--------------------|-------------------------------------------------------------|
| Table and fields   | Custom table, student details, and Branch choices.          |
| Roles and identity | Test user and roles bb1–bb4.                                |
| Access rules       | Read condition, script check, and Create/Write/Delete ACLs. |
| Testing            | Impersonation and verification of permitted actions.        |

## Step 3: Idea Prioritization

### Brainstormed Ideas & Prioritization

| Idea                                                    | Importance | Feasibility | Priority |
|---------------------------------------------------------|------------|-------------|----------|
| Restrict Read access using Branch = EEE and role checks | High       | High        | 1        |
| Create separate ACLs for Create, Write, and Delete      | High       | High        | 2        |
| Create the custom Institution Details table and fields  | High       | High        | 3        |
| Create and assign custom roles bb1–bb4                  | High       | High        | 4        |
| Test permissions using different role combinations      | High       | High        | 5        |
| Verify administrator access and overall ACL behavior    | High       | Medium      | 6        |

### Selected Idea

Develop a **Script-Controlled ACL solution in ServiceNow** to restrict record access based on the Branch field and user roles. The solution uses the Institution Details table, a Read ACL with the Branch = EEE condition and role-checking script, and separate Create, Write, and Delete ACLs. The team will validate the permissions by testing users with different role combinations.

Figure 3: Selected solution flow for script-controlled record access.

### Expected Outcome

The project is intended to demonstrate how role-based ACLs and a field-based condition can work together to protect student records. Authorized users receive the permissions assigned to their roles, while access to EEE records is restricted according to the Read ACL configuration.

**Note:** The script's role checks and the Branch data condition are separate parts of the Read ACL. The configured behavior should be confirmed in the ServiceNow instance, since other applicable ACL rules may also affect access.